



Sistemas embebidos

Laboratorio 2 – TouchPad, WiFi y Webserver

Integrantes del grupo:

- Agustina Bacigalupe
- Pierina Borsieri
- Adrian Tesore
- Sofia Nicoletti

Docentes: Nicolás Calarco y Pablo

29/05/2025

Objetivo:

Ya con algunas nociones básicas del IDE y la placa de desarrollo del curso, se seguirá avanzando en el control del hardware, se conectará el dispositivo a una red WiFi y generará su propia red, así como también se montará un pequeño servidor web en el dispositivo. Se comenzará a utilizar la herramienta brindada por Espressif para la configuración de registros de periféricos a través de sus APIs (Interfaz de Programación de Aplicaciones). Con esta herramienta, se configurarán puertos de entrada para el uso de botones, comunicación WiFi y servidor web.

Parte 1:

En esta primera parte del laboratorio se implementó el uso de la placa de expansión touchpad. El objetivo de esta etapa fue reutilizar las librerías previamente desarrolladas en el Laboratorio 1 e integrarlas con el nuevo sistema de entrada táctil.

Posteriormente, se desarrolló una nueva librería dedicada al manejo del TouchPad. Esta se encargó de inicializar los sensores táctiles y de implementar una lectura continua de los valores proporcionados por los sensores.

Cada botón del TouchPad fue configurado para ejecutar una acción diferente sobre el LED RGB. En concreto, se asignaron funciones para aumentar o disminuir el brillo, hacer parpadear el color actual, y cambiar el color del LED a rojo, verde o azul. Este comportamiento fue programado dentro de un bucle infinito en la función `tp_read()`, la cual mapea la entrada de cada botón a su acción correspondiente sobre el LED.

El proyecto quedó estructurado con tres módulos principales: `delay`, `led` y `touch_pad`, junto con el archivo `main.c`, que se encarga de inicializar todos los componentes y arrancar la ejecución del sistema.

Por último, se eliminó el `freertos`, esto se realizó retirando los `include`, agregando el `delay` previamente mencionado (utilizando en el laboratorio anterior) y deshabilitando el `watchdog`.

Es pertinente mencionar, que no se modificó nada en la placa base, para poder conectar la placa de expansión. Solo se tuvo en cuenta el esquemático del touchpad para así poder modificar el código utilizado en este parte.

A la hora de utilizarlo, los problemas que se presentan en la placa de extensión son que al conectar la placa de audio uno de los botones de la placa de extensión deja de funcionar.

Pseudocódigo:

Si s_strip no está inicializado

Mostrar error "tp_set_led_strip() NO fue llamado antes de
tp_read()" Salir del procedimiento

FinSi

Declarar touch_value (valor del sensor)

Declarar now (tiempo actual)

Esperar 100 ms

Mostrar "Lectura del sensor táctil:"

Mientras verdadero

Obtener tiempo actual (en ms)

Para cada botón (i) del arreglo de botones

Leer valor táctil de botón[i] y guardarlo en touch_value

Si touch_value supera umbral y ha pasado tiempo de rebote
(DEBOUNCE_MS)

Actualizar last_time[i] a tiempo actual

Según el botón[i]

Caso VOL_UP:

Aumentar brillo en 20 (máximo 255)

Mostrar brillo actualizado

Configurar color del LED con el nuevo brillo

Caso VOL_DOWN:

Disminuir brillo en 20 (mínimo 0)

Mostrar brillo actualizado

Configurar color del LED con el nuevo brillo

Caso PLAY/PAUSE:

Mostrar "PLAY/PAUSE: parpadeo"

Repetir dos veces:

Apagar LED

Esperar 200 ms
Encender LED con el color actual y brillo
Esperar 200 ms

Caso RECORD (ROJO):
Mostrar "RECORD: color Rojo"
Configurar color del LED a rojo con el brillo actual

Caso PHOTO (VERDE):
Mostrar "PHOTO: color Verde"
Configurar color del LED a verde con el brillo actual

Caso NETWORK (AZUL):
Mostrar "NETWORK: color Azul"
Configurar color del LED a azul con el brillo actual

Otro:
No hacer nada

FinSegún

FinSi

FinPara

Esperar 100 ms
FinMientras

FinAlgoritmo

Parte 2

- Funcion `esp_netif_init()`;
Dicha función es la encargada de inicializar el sistema de interfaces de red del ESP32. A la hora de crear cualquier interfaz de red, es un paso que no debemos saltar, ya sea creandola como acces point o como statios.
Internamente esta funcion configura las estructuras de datos y mecanismos necesarios para manejar conexiones de red. En resumen, `esp_netif_init()` actúa como el punto de partida de toda configuración de red. Es imprescindible incluirla antes de realizar cualquier acción relacionada con WiFi.

- Funcion `esp_event_loop_create_default()`;

Esta función tiene como objetivo crear y configurar el bucle de eventos predeterminados del sistema. Dicho bucle es el mecanismo principal por el cual se procesan eventos asincrónicos, como los generados por la conectividad WiFi, sensores, timers, etc

Centrándonos en el caso de redes WiFi, eventos importantes, como lo es por ejemplo la conexión cuando es de forma exitosa o no (conexión o desconexión), la misma se notifica mediante el Event Loop, es decir, para poder capturar un evento así y poder realizar acciones correspondientes, es indispensable haber creado previamente el ya mencionado bucle de eventos.

Configuraciones del api del Wifi- El api del WiFi cuenta con las siguientes configuraciones:

- Station mode (aka STA mode or Wi-Fi client mode). ESP32-S2 connects to an access point.
- AP mode (aka Soft-AP mode or Access Point mode). Stations connect to the ESP32-S2.
- Combined AP-STA mode (ESP32-S2 is concurrently an access point and a station connected to another access point).
- Various security modes for the above (WPA, WPA2, WPA3, etc.)
- Scanning for access points (active & passive scanning).
- Promiscuous mode for monitoring of IEEE802.11 Wi-Fi packets.

```
I (462) wifi_init: WiFi IRAM OP enabled
I (472) wifi_init: WiFi RX IRAM OP enabled
I (482) phy_init: phy_version 2620,f8f9319,feb 6 2025,14:35:09
I (522) wifi:mode : softAP (7c:df:a1:00:ab:01)
I (582) wifi:Total power save buffer number: 16
I (582) wifi:Init max length of beacon: 752/752
I (582) wifi:Init max length of beacon: 752/752
I (592) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
I (592) wifi softAP: wifi_init_softap finished. SSID:myssid password:mypassword channel:1
I (602) main_task: Returned from app_main()
I (37312) wifi:new: <1,0>, old: <1,1>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (37312) wifi:station: fa:4d:b9:3e:00:59 join, AID=1, bgn, 20
I (37332) wifi softAP: station fa:4d:b9:3e:00:59 join, AID=1
I (38492) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
I (38572) wifi:removing station <10:51:07:79:77:fa> after unsuccessful auth/assoc, AID = 0
W (38572) wifi:rm mis
I (38572) wifi:new: <1,0>, old: <1,0>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (40532) wifi:cba-addidx:2 (ifx:1, fa:4d:b9:3e:00:59), tid:0, ssn:2, winSize:64
I (45702) wifi:removing station <10:51:07:79:77:fa> after unsuccessful auth/assoc, AID = 0
W (45702) wifi:rm mis
I (45702) wifi:new: <1,0>, old: <1,0>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (63642) wifi:cba-addidx:3 (ifx:1, fa:4d:b9:3e:00:59), tid:1, ssn:0, winSize:64
I (63672) wifi:cba-addidx:4 (ifx:1, fa:4d:b9:3e:00:59), tid:5, ssn:0, winSize:64
I (74412) wifi:station: fa:4d:b9:3e:00:59 leave, AID = 1, reason = 8, bss_flags is 8948819, bss:0x3ffdb3394
I (74412) wifi:new: <1,0>, old: <1,0>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (74422) wifi:cba-delidx:2, tid:0
I (74422) wifi:cba-delidx:3, tid:1
I (74422) wifi:cba-delidx:4, tid:5
I (74422) wifi softAP: station fa:4d:b9:3e:00:59 leave, AID=1, reason=8
I (75142) wifi:new: <1,0>, old: <1,0>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (75142) wifi:station: fa:4d:b9:3e:00:59 join, AID=1, bgn, 20
I (75162) wifi softAP: station fa:4d:b9:3e:00:59 join, AID=1
I (77492) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2
I (77492) wifi:cba-addidx:2 (ifx:1, fa:4d:b9:3e:00:59), tid:5, ssn:0, winSize:64
I (79382) wifi:cba-addidx:3 (ifx:1, fa:4d:b9:3e:00:59), tid:0, ssn:2, winSize:64
I (80342) wifi:cba-addidx:4 (ifx:1, fa:4d:b9:3e:00:59), tid:1, ssn:0, winSize:64
I (98632) wifi:station: fa:4d:b9:3e:00:59 leave, AID = 1, reason = 8, bss_flags is 691315, bss:0x3ffe37a8
I (98632) wifi:new: <1,0>, old: <1,0>, ap: <1,0>, sta: <255,255>, prof:1, snd_ch_cfg:0x0
I (98642) wifi:cba-delidx:3, tid:0
I (98642) wifi:cba-delidx:4, tid:1
I (98642) wifi:cba-delidx:2, tid:5
I (98652) wifi softAP: station fa:4d:b9:3e:00:59 leave, AID=1, reason=8
```

Conexión y desconexión de 2 máquinas a la red AP de esp.

Todo es configurable en la esp32 mediante la siguiente typedef que nos brinda `wifi_config_t`:

Por un lado tenemos `authmode` para definir el tipo de red (abierta, o con contraseña WPA2...) despues tenemos `ssid_hidden` que nos da la posibilidad de ocultar la red pasandole como parametro 1, en caso de no querer ocultar la red le pasamos 0;

```
wifi_config_t wifi_config = {  
    .ap = {  
        .ssid = EXAMPLE_ESP_WIFI_SSID,  
        .ssid_len = strlen(EXAMPLE_ESP_WIFI_SSID),  
        .channel = EXAMPLE_ESP_WIFI_CHANNEL,  
        .password = EXAMPLE_ESP_WIFI_PASS,  
        .max_connection = EXAMPLE_MAX_STA_CONN,  
#ifdef CONFIG_ESP_WIFI_SOFTAP_SAE_SUPPORT  
        .authmode = WIFI_AUTH_WPA3_PSK,  
        .sae_pwe_h2e = WPA3_SAE_PWE_BOTH,  
#else /* CONFIG_ESP_WIFI_SOFTAP_SAE_SUPPORT */  
        .authmode = WIFI_AUTH_WPA2_PSK,  
#endif  
        .ssid_hidden = hidden,  
        .pmf_cfg = {  
            .required = true,  
        },  
    },  
};
```

```
I (603) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (613) wifi:<ba-add>idx:0 (ifx:0, 7c:39:85:03:18:90), tid:6, ssn:2, winSize:64
I (623) wifi:<ba-add>idx:1 (ifx:0, 7c:39:85:03:18:90), tid:0, ssn:0, winSize:64
I (633) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (1643) esp_netif_handlers: sta ip: 10.4.96.55, mask: 255.255.240.0, gw: 10.4.100.1
I (1643) wifi station: got ip:10.4.96.55
I (1643) wifi station: connected to ap SSID:caliope password:sinlugar
I (1643) main_task: Returned from app_main()
```

Conexión del equipo a la red Caliope

Como se puede observar en la imagen se nos asignó una ip: 10.4.96.55 que tiene como mascara de red 255.255.240 y está conectado al ssid: caliope con password: sinlugar.

Diferencia con AP: En el modo STA, el ESP se conecta a una red WiFi ya existente como un cliente, y recibe una dirección IP del router mediante DHCP. En cambio, en el modo AP el ESP crea su propia red WiFi y actúa como servidor, permitiendo que otros dispositivos se conecten a él directamente. Es decir, en STA se conecta a una red, mientras que en AP crea la red.

Pseudocódigo como implementar AP:

Crear configuración de red para modo SoftAP (punto de acceso)

Inicializar configuración de Wi-Fi por defecto

Comprobar que la inicialización de Wi-Fi se realizó correctamente

Registrar el manejador de eventos para Wi-Fi

Comprobar que el registro se realizó correctamente

Configurar parámetros del Wi-Fi (wifi_config):

- Nombre de red (SSID)
- Longitud del SSID
- Canal de Wi-Fi
- Contraseña
- Máximo número de conexiones
- Modo de autenticación: WPA3_PSK o WPA2_PSK según configuración
- Si es red oculta: establecer ocultamiento según parámetro

'hidden'

- Configurar protección PMF como obligatoria

Si openWifi es igual a 1

Establecer modo de autenticación a abierto (sin contraseña)
FinSi

Configurar modo Wi-Fi a punto de acceso (AP)
Aplicar configuración de Wi-Fi usando los parámetros definidos
Iniciar servicio de Wi-Fi
Comprobar que el inicio se realizó correctamente

Mostrar mensaje de que la configuración del SoftAP se completó
(Incluir SSID, contraseña y canal)

FinAlgoritmo

Pseudocódigo implementación STA:

Inicializar NVS (almacenamiento no volátil)

Si NVS está corrupta o con nueva versión

Borrar NVS

Inicializar NVS nuevamente

FinSi

Verificar que NVS se inicializó correctamente

Crear grupo de eventos para conexión y falla (s_wifi_event_group)

Inicializar red y loop de eventos (modo estación WiFi)

Inicializar configuración WiFi por defecto

Verificar que la inicialización fue correcta

Registrar manejadores de eventos:

- Evento WiFi (cualquier ID)
- Evento de obtención de IP (STA_GOT_IP)

Verificar registros correctos

Configurar conexión WiFi:

- Modo autenticación: WPA2_PSK
- Copiar ssid recibido al campo de configuración
- Copiar password recibido al campo de configuración

Configurar WiFi en modo estación (WIFI_MODE_STA)

Aplicar configuración con SSID y password

Iniciar WiFi

Mostrar mensaje "wifi_init_station finalizado"

Esperar evento de conexión exitosa o falla

Si se recibió el evento WIFI_CONNECTED_BIT

Mostrar mensaje "Conectado a SSID"

Devolver ESP_OK (éxito)

Sino si se recibió el evento WIFI_FAIL_BIT

Mostrar mensaje "Fallo al conectar a SSID"

Devolver ESP_FAIL (error)

FinSi

Devolver ESP_FAIL (error por tiempo agotado o desconocido)

FinAlgoritmo

Parte 3:

Pseudocódigo

Declarar server como manejador HTTP inicializado en NULL

Inicializar almacenamiento NVS

Verificar que se inicializó correctamente

Inicializar red (esp_netif)

Verificar que se inicializó correctamente

Crear bucle de eventos por defecto

Verificar que se creó correctamente

Establecer conexión de red (Wi-Fi o Ethernet)

Verificar que se realizó correctamente

Registrar manejadores de eventos para:

- Cuando se obtiene IP por Wi-Fi o Ethernet -> llamar a connect_handler

- Cuando se pierde conexión por Wi-Fi o Ethernet -> llamar a disconnect_handler

(Solo si no se está en entorno Linux)

Iniciar servidor web y guardar el manejador en server

Mientras server esté activo

Esperar 5 segundos

FinMientras

FinAlgoritmo

ESP32-S2 Kaluga-1 TouchPad

Enter value:

Send

Dato recibido: a

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF
I (3317) wifi:security: WPA2-PSK, phy: bgn, rssi: -57
I (3317) wifi:pm start, type: 1
I (3317) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (3327) wifi:<ba-add>idx:0 (ifx:0, 7c:39:85:03:1b:70), tid:6, ssn:2, winSize:64
I (3367) wifi:<ba-add>idx:1 (ifx:0, 7c:39:85:03:1b:70), tid:0, ssn:0, winSize:64
I (3377) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (4377) esp_netif_handlers: example_netif_sta ip: 10.4.96.62, mask: 255.255.240.0, gw: 10.4.100.1
I (4377) example_connect: Got IPv4 event: Interface "example_netif_sta" address: 10.4.96.62
I (4407) example_connect: Got IPv6 event: Interface "example_netif_sta" address: fe80:0000:0000:0000:7edf:a1ff:fe00:ab00, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (4407) example_common: Connected to example_netif_sta
I (4407) example_common: - IPv4 address: 10.4.96.62,
I (4417) example_common: - IPv6 address: fe80:0000:0000:0000:7edf:a1ff:fe00:ab00, type: ESP_IP6_ADDR_IS_LINK_LOCAL
I (4427) example: Starting server on port: '80'
I (4437) example: Registering URI handlers
I (10697) example: Dato recibido: a
I (13417) example: Dato recibido: a
[]
```

Envío de dato y su vista en la consola

¿Cómo haría para enviar información al equipo?

Para enviar información al equipo desde un cliente externo, se implementó como solución la creación de una página web alojada dentro del propio ESP32-S2. Esta página actúa como una interfaz visual para el usuario, permitiéndole interactuar con el sistema a través del navegador.

La comunicación entre el usuario y el dispositivo se realiza mediante el protocolo HTTP. En la página web se incluye un formulario donde se pone un texto el cual será enviado al equipo mediante una solicitud HTTP de tipo POST. De esta forma, el ESP32-S2 puede recibir la información enviada por el usuario, procesarla y ejecutar la acción correspondiente, que en nuestro caso es encender el led de color verde.

Para que el ESP pueda instalar la página web, fue necesario incluir el archivo HTML como parte del firmware del dispositivo. Esto se logra mediante una funcionalidad del entorno de desarrollo que permite incorporar archivos estáticos en la compilación, asegurando que estén disponibles en la memoria del microcontrolador al momento de ejecutar el programa.

Una vez que el servidor web del ESP está funcionando, el usuario puede acceder a la interfaz desde cualquier navegador conectado a la misma red. Al interactuar con la página, la información se transmite al equipo, el cual la interpreta y responde en consecuencia. Esta estrategia resulta eficiente, ya que no requiere software adicional por parte del usuario, y permite una comunicación clara y directa con el sistema embebido.

¿Qué datos de un sistema IoT cree relevante para la persistir en el equipo?

Los datos que resultan relevantes para persistir en el equipo son aquellos que permiten mantener la continuidad del servicio y garantizar la seguridad ya sea si el dispositivo se reinicia o se pierde la conexión temporalmente.

Los principales tipos de datos a persistir serían:

En cuanto a la configuración del sistema, las credenciales de red Wifi, la configuración de modo de operación AP o STA, la dirección IP si es que es fija y el puerto de comunicación. Y los parámetros del servidor web como el nombre de red, contraseña y configuraciones de acceso.

Los datos de autenticación y seguridad: configurar claves API, tokens que permitan la autenticación con otros servicios. Y configuraciones de cifrado o listas de control de acceso.

Los estados importantes del dispositivo ya sea los últimos estados de los sensores o actuadores críticos como por ejemplo la configuración de un led.

Los logs y eventos críticos, osea los registros de eventos o errores ocurridos antes de un reinicio inesperado.

También, archivos .html incorporados en la memoria flash del dispositivo para ser servicios por el servidor web.